Reto 2 — Postman (DummyJSON) + SQL (Sakila)

Master Profesional en QA y Automatización de Pruebas

Adriana Troche Robles

23 de abril de 2026

1. Introducción 2
1.1 Contexto 2
1.2 Objetivos 2
1.3 Alcance 2
2. Sección 1 — Postman (DummyJSON) 2
2.1 Setup del entorno Postman 2
2.2 Incidencia detectada: credenciales del enunciado 3
2.3 Ejercicio 1 — Login y validación inicial 5
2.4 Ejercicio 2 — Validación de sesión (token en uso) 6
3.5 Ejercicio 3 — Manejo de token 6
2.6 Ejercicio 4 — Validación de lista con lógica 7
2.7 Ejercicio 5 — Flujo automático (retry login) 8
3. Sección 2 — SQL (Sakila) 10
3.1 Elección del motor: SQLite 10
3.2 Ejercicios 1 a 5 10
Ejercicio 1 (básico) — Nombre y apellido de actores 10
Ejercicio 2 (intermedio) — Películas con duración > 100 minutos 11
Ejercicio 3 (intermedio) — Película + categoría (JOIN) 12
Ejercicio 4 (difícil) — Número de películas por categoría 13
Ejercicio 5 (difícil) — Por cliente: total de rentas y total pagado 14
4. Cómo validar el entregable 14
4.1 Postman 14
4.2 SQL 15
Opción A — DBeaver (cliente gráfico, utilizado para el desarrollo) 15
Opción B — Terminal (sqlite3, cero instalación) 16
4.3 Validación cruzada 16
5. Conclusiones 16
6. Referencias 17

1. Introducción

1.1 Contexto

El Reto 2 del Master evalúa la capacidad para validar APIs REST utilizando **Postman** (con tests y post-scripts, manejo de variables dinámicas y lógica condicional) y para consultar bases de datos mediante **SQL** (con JOINs, agregaciones y ordenamientos). Está organizado en 10 ejercicios (5 por sección) de dificultad progresiva: básico → intermedio → difícil.

1.2 Objetivos

- Construir una colección Postman reutilizable, con variables de entorno y sin valores hardcodeados.
- Implementar tests automatizados que cubran los criterios de cada ejercicio.
- Demostrar manejo de lógica condicional y flujo controlado entre requests (`setNextRequest`).
- Escribir las 5 consultas SQL equivalentes sobre la base de datos Sakila, validadas contra datos reales.
- Entregar un paquete autocontenido, fácilmente evaluable por el profesor.

1.3 Alcance

El entregable es una carpeta con la siguiente estructura:

```
reto2-dummyjson-sakila/
├── INFORME-RETO2.docx                                  ← este informe
├── README.md                                         ← instrucciones rápidas
├── postman/
│   ├── reto2-dummyjson.postman_collection.json     ← colección (5
requests)
│   └── reto2-dummyjson.postman_environment.json    ← environment (7
variables)
└── sql/
    ├── reto2-sakila.sql                            ← 5 queries comentadas
    └── sakila.db                                   ← base SQLite lista
para usar
```

Se entrega comprimida en `.zip`.

---

2. Sección 1 — Postman (DummyJSON)

URL base: `https://dummyjson.com`

2.1 Setup del entorno Postman

Se creó un **environment** (`Reto2 - DummyJSON`) con las siguientes variables, para cumplir la regla *"No hardcodees valores si puedes reutilizarlos"*:

Variable	Tipo	Propósito
`base_url`	default	URL del servicio (`https://dummyjson.com`)
`username`	default	Usuario del login (reutilizado en asserts)
`password`	secret	Contraseña del login
`token`	secret	`accessToken` devuelto por `POST /auth/login`
`session_valid`	default	`true`/`false` según resultado de `GET /auth/me`
`expired_token`	default	Bandera del flujo de retry (Ejercicio 5)
`expensive_count`	default	Cantidad de productos con `price > 50` (Ejercicio 4)

2.2 Incidencia detectada: credenciales del enunciado

El enunciado indica usar:

```
{ "username": "kminchelle", "password": "0lelplR" }
```

Estas credenciales ya no son válidas. DummyJSON actualizó su sistema de autenticación y la API devuelve `400 Invalid credentials` al usarlas. Se verificó en vivo:

POST https://dummyjson.com/auth/login Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "username":"kminchelle",
3   "password":"0lelplR"
4
5 }
```

Body Cookies Headers 21 Test Results 400 Bad Request 1.38 s 989 B Save Response

{} JSON Preview Debug with AI

```
1 {
2   "message": "Invalid credentials"
3 }
4
```

POST https://dummyjson.com/auth/login Send

Docs Params Authorization Headers 11 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "username":"emilys",
3   "password":"emilyspass"
4
5 }
```

Body Cookies 2 Headers 23 Test Results 200 OK 432 ms 2.3 KB Save Response

{} JSON Preview Visualize


```
1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
      eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlseXMiLCJlbWFpbCI6ImVtaWx5LmpvaG5zb25AeC5kdW1teWpzb24uY29tIiwiZmly
      c3ROYW1lIjoiRW1pbHkiLCJsYXN0TmFtZSI6IkpvaG5zb24iLCJnZW5kZXIiOiJmZW1hbGUiLCJpbWFnZSI6Imh0dHBzOi8v
      ZHVtbXlqc29uLmNvbS9pY29uL2VtaWx5cy8xMjgiLCJpYXQiOjE3NzY5OTg5MTEsImV4cCI6MTc3NzAwMjUxMX0.
      wT7FZ0aLodR3QBqq8oC5NHXGHK9DD2EhHQsCDDvZHFU",
3   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
      eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlseXMiLCJlbWFpbCI6ImVtaWx5LmpvaG5zb25AeC5kdW1teWpzb24uY29tIiwiZmly
      c3ROYW1lIjoiRW1pbHkiLCJsYXN0TmFtZSI6IkpvaG5zb24iLCJnZW5kZXIiOiJmZW1hbGUiLCJpbWFnZSI6Imh0dHBzOi8v
      ZHVtbXlqc29uLmNvbS9pY29uL2VtaWx5cy8xMjgiLCJpYXQiOjE3NzY5OTg5MTEsImV4cCI6MTc3OTU5MDkxMX0.
      Rk7GDfyzcp-d5oC4I91z9kqYjarpjs4AJckd5xXJk_w",
4   "id": 1,
5   "username": "emilys",
6   "email": "emily.johnson@x.dummyjson.com",
7   "firstName": "Emily",
8   "lastName": "Johnson",
9   "gender": "female",
10  "image": "https://dummyjson.com/icon/emilys/128"
11 }
12
```


Se usaron las credenciales vigentes (`emilys / emilyspass`). La lógica evaluada del reto (tests, condicionales, retry) es idéntica con cualquier usuario válido; el cambio solo afecta al par usuario/contraseña.

Observación adicional: la API actual devuelve el token como `accessToken` (no `token`), lo cual coincide exactamente con lo que pide el Ejercicio 3 (*“guarda accessToken en variable token”*).

2.3 Ejercicio 1 — Login y validación inicial

Endpoint: `POST /auth/login`

Body:

```
{
  "username": "{{username}}",
  "password": "{{password}}"
}
```

Tests implementados (pestaña Tests):

```
pm.test('Status code es 200', function () {
    pm.response.to.have.status(200);
});

let jsonData;
pm.test('El body se parsea correctamente como JSON', function () {
    jsonData = pm.response.json();
    pm.expect(jsonData).to.be.an('object');
});

pm.test('Existe accessToken en la respuesta', function () {
    pm.expect(jsonData).to.have.property('accessToken');
    pm.expect(jsonData.accessToken).to.be.a('string').and.not.empty;
});

pm.test('Username coincide con el enviado en el login', function () {
    pm.expect(jsonData.username).to.eql(pm.environment.get('username'));
});

// Persistimos el token para los siguientes requests
if (jsonData && jsonData.accessToken) {
    pm.environment.set('token', jsonData.accessToken);
}
```

Cobertura de criterios:

Criterio del enunciado	Implementación
Status code = 200	`pm.response.to.have.status(200)`
Existe accessToken	`expect(jsonData).to.have.property('accessToken')`
Username correcto	`expect(jsonData.username).to.eql(<username env var>)`
Parseo correcto del JSON	`jsonData = pm.response.json()` + `expect(...).to.be.an('object')`

2.4 Ejercicio 2 — Validación de sesión (token en uso)

Endpoint: `GET /auth/me` **Header:** `Authorization: Bearer {{token}}`

Tests con lógica condicional:

```
const status = pm.response.code;

if (status === 200) {
    pm.environment.set('session_valid', 'true');
    const data = pm.response.json();

    pm.test('Status 200', function () {
        pm.response.to.have.status(200);
    });

    pm.test('Existe email en la respuesta', function () {
        pm.expect(data).to.have.property('email');
        pm.expect(data.email).to.be.a('string').and.not.empty;
    });

    pm.test('Username coincide con el enviado en el login', function () {
        pm.expect(data.username).to.eql(pm.environment.get('username'));
    });
} else {
    pm.environment.set('session_valid', 'false');

    pm.test('Sesión inválida registrada (session_valid = false)', function ()
{
        pm.expect(pm.environment.get('session_valid')).to.eql('false');
    });
}
```

Cobertura:

- Al menos **1 test**: se cumplen 3 (status, email, username) en el camino feliz y 1 en el camino de error.
- Al menos **1 condicional**: `if (status === 200) { ... } else { ... }` que actualiza la variable `session_valid` dinámicamente.

3.5 Ejercicio 3 — Manejo de token

Este ejercicio se implementó como un request independiente (misma llamada a `POST /auth/login`) con tests focalizados en la persistencia del token, para hacer explícito el cumplimiento del criterio del enunciado ("*el test debe fallar si no se guarda correctamente*").

```
pm.test('Status code es 200', function () {
    pm.response.to.have.status(200);
});
```

```
const jsonData = pm.response.json();

// Guardar accessToken en la variable 'token'
pm.environment.set('token', jsonData.accessToken);

pm.test("accessToken fue guardado en la variable 'token'", function () {
    const saved = pm.environment.get('token');
    pm.expect(saved, 'variable token').to.not.be.null;
    pm.expect(saved, 'variable token').to.not.be.undefined;
    pm.expect(saved).to.be.a('string').and.not.empty;
});

pm.test('El token guardado coincide con el accessToken de la respuesta',
function () {
    pm.expect(pm.environment.get('token')).to.eql(jsonData.accessToken);
});
```

Si el `accessToken` no estuviera presente en la respuesta o la asignación fallara, las expectativas `not.be.null`, `not.be.undefined` y `not.empty` harían fallar el test, exactamente como pide el enunciado.

2.6 Ejercicio 4 — Validación de lista con lógica

Endpoint: `GET /products?limit=100`

Se agrega el parámetro `?limit=100` para obtener la muestra completa del catálogo en una sola llamada (DummyJSON devuelve 30 productos por defecto; el máximo por página es 100).

Tests con iteración y condicional:

```
pm.test('Status 200', function () {
    pm.response.to.have.status(200);
});

const body = pm.response.json();
const products = body.products || [];

pm.test('Se recibió un array de productos no vacío', function () {
    pm.expect(products).to.be.an('array').that.is.not.empty;
});

pm.test('Todos los productos tienen price > 0', function () {
    products.forEach(function (p) {
        pm.expect(p.price, 'price de ' +
p.title).to.be.a('number').and.greaterThan(0);
    });
});
```

```
pm.test('Todos los productos tienen stock >= 0', function () {
    products.forEach(function (p) {
        pm.expect(p.stock, 'stock de ' +
p.title).to.be.a('number').and.at.least(0);
    });
});

// Lógica condicional: price > 100 → debe existir discountPercentage
pm.test('Productos con price > 100 tienen discountPercentage', function () {
    const caros = products.filter(function (p) { return p.price > 100; });
    caros.forEach(function (p) {
        pm.expect(p).to.have.property('discountPercentage');
        pm.expect(p.discountPercentage).to.not.be.null;
        pm.expect(p.discountPercentage).to.not.be.undefined;
    });
});

// Contar productos con price > 50 y guardar
const expensiveCount = products.filter(function (p) { return p.price > 50;
}).length;
pm.environment.set('expensive_count', expensiveCount);

pm.test('expensive_count guardado (productos con price > 50)', function () {

pm.expect(Number(pm.environment.get('expensive_count'))).to.eql(expensiveCoun
t);
});
```

-

2.7 Ejercicio 5 — Flujo automático (retry login)

Este es el ejercicio más complejo porque involucra control de flujo entre requests.

Estrategia adoptada:

1. El request `Ej 5 - Auth Me (retry)` llama a `GET /auth/me` usando el `token` actual.
2. Si la respuesta es **401** → se marca `expired_token = true` y se redirige al request de login con `pm.execution.setNextRequest('Ej 1 - Login (básico)')`.
3. El Login (Ej 1) refresca el token normalmente. Al final, detecta que viene de un retry (`expired_token === 'true'`), resetea la bandera y llama `setNextRequest(null)` para **detener el runner**, evitando un bucle infinito.
4. Si la respuesta es **200** → se marca `expired_token = false`, `session_valid = true` y el flujo continúa su orden normal.

Código del Ejercicio 5:

```
const status = pm.response.code;

pm.test('Status esperado (200 o 401)', function () {
    pm.expect([200, 401]).to.include(status);
});

if (status === 401) {
    console.warn('[Retry] Token inválido/expirado. Redirigiendo a Login...');
    pm.environment.set('expired_token', 'true');
    pm.environment.set('session_valid', 'false');

    pm.test('Se marcó expired_token = true', function () {
        pm.expect(pm.environment.get('expired_token')).to.eql('true');
    });

    pm.execution.setNextRequest('Ej 1 - Login (básico)');
} else if (status === 200) {
    pm.environment.set('expired_token', 'false');
    pm.environment.set('session_valid', 'true');

    const data = pm.response.json();

    pm.test('Sesión vigente: existe email', function () {
        pm.expect(data).to.have.property('email');
    });
    // Sin setNextRequest → el runner continúa con el orden por defecto.
}
```

Bloque anti-bucle en el Login (Ej 1):

```
if (pm.environment.get('expired_token') === 'true') {
    console.log('[Retry] Login de refresh exitoso. Nuevo token guardado.');
    pm.environment.set('expired_token', 'false');
    pm.execution.setNextRequest(null); // detener el runner
}
```

Cómo probar el retry manualmente (incluido también en el README):

1. Correr la colección completa → todos los tests en verde.
2. Editar el `token` en el environment y poner un valor inválido (por ejemplo, `invalid`).
3. Correr **solo el Ej 5** → recibirá 401 → el script redirige al Login → refresca el token → detiene.
4. Volver a correr el Ej 5 → ahora devuelve 200 y el flujo es normal.

3. Sección 2 — SQL (Sakila)

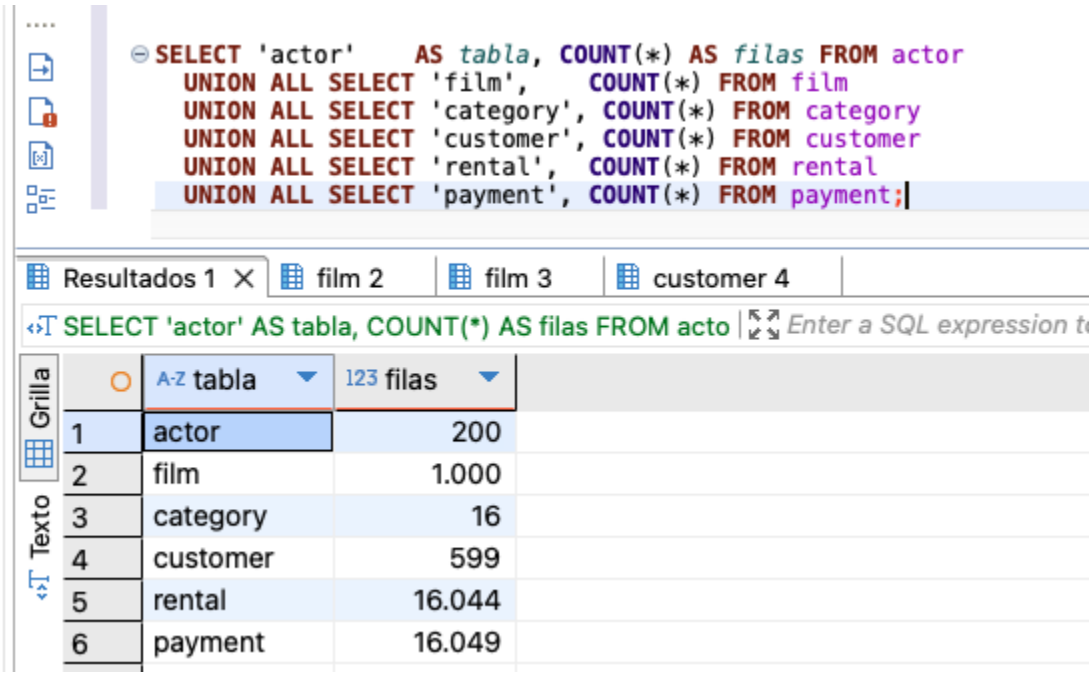
3.1 Elección del motor: SQLite

La base de datos original de Sakila es para MySQL. En este entorno se optó por **SQLite** por las siguientes razones:

Criterio	SQLite	MySQL
Instalación	Ya viene con macOS (`sqlite3`)	Requiere Homebrew/Docker + configuración
Compatibilidad con el reto	100% — usa solo SQL estándar	100%
Schema de Sakila	Port oficial disponible en GitHub, idéntico	Original
Reproducibilidad	Archivo `.db` autocontenido, sin servicio separado	Requiere servicio corriendo

Las queries del reto usan exclusivamente **SQL estándar** (`SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `COUNT`, `SUM`, `ORDER BY`, `ROUND`) → se ejecutan tal cual en MySQL Sakila sin cambios.

Verificación inicial del estado de la base:

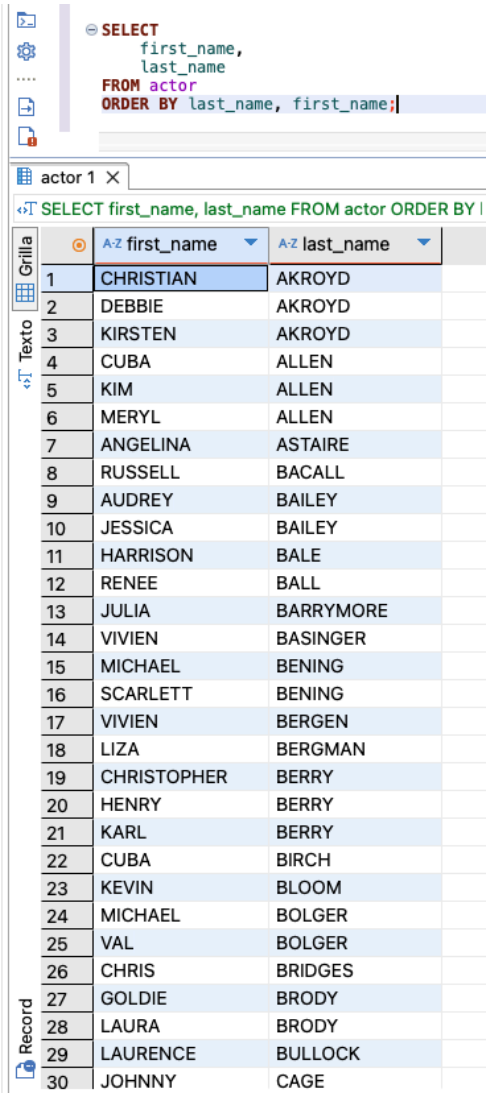


3.2 Ejercicios 1 a 5

Ejercicio 1 (básico) — Nombre y apellido de actores

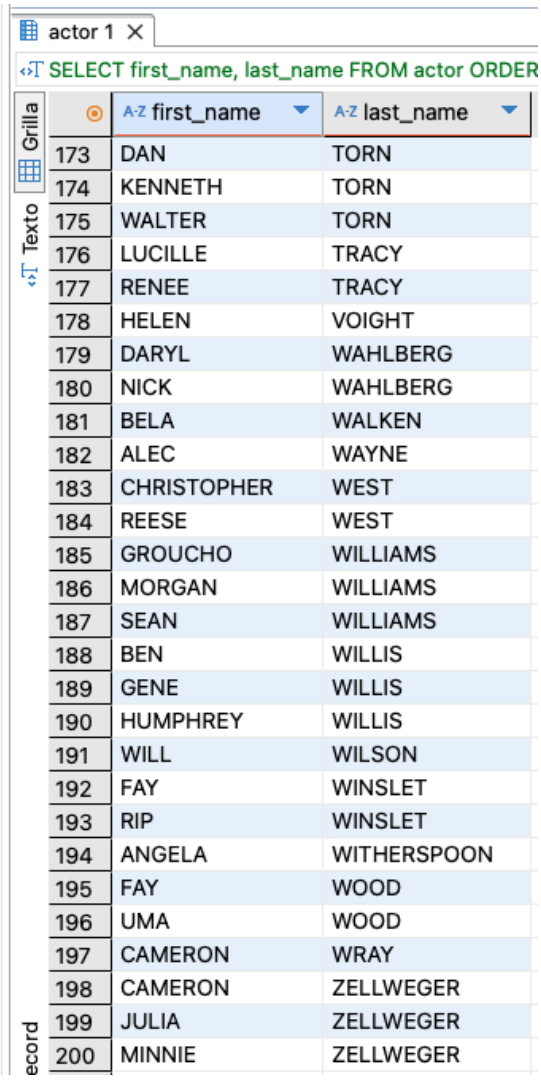
```
SELECT
    first_name,
    last_name
FROM actor
ORDER BY last_name, first_name;
```

Resultado:



	first_name	last_name
1	CHRISTIAN	AKROYD
2	DEBBIE	AKROYD
3	KIRSTEN	AKROYD
4	CUBA	ALLEN
5	KIM	ALLEN
6	MERYL	ALLEN
7	ANGELINA	ASTAIRE
8	RUSSELL	BACALL
9	AUDREY	BAILEY
10	JESSICA	BAILEY
11	HARRISON	BALE
12	RENEE	BALL
13	JULIA	BARRYMORE
14	VIVIEN	BASINGER
15	MICHAEL	BENING
16	SCARLETT	BENING
17	VIVIEN	BERGEN
18	LIZA	BERGMAN
19	CHRISTOPHER	BERRY
20	HENRY	BERRY
21	KARL	BERRY
22	CUBA	BIRCH
23	KEVIN	BLOOM
24	MICHAEL	BOLGER
25	VAL	BOLGER
26	CHRIS	BRIDGES
27	GOLDIE	BRODY
28	LAURA	BRODY
29	LAURENCE	BULLOCK
30	JOHNNY	CAGE

...



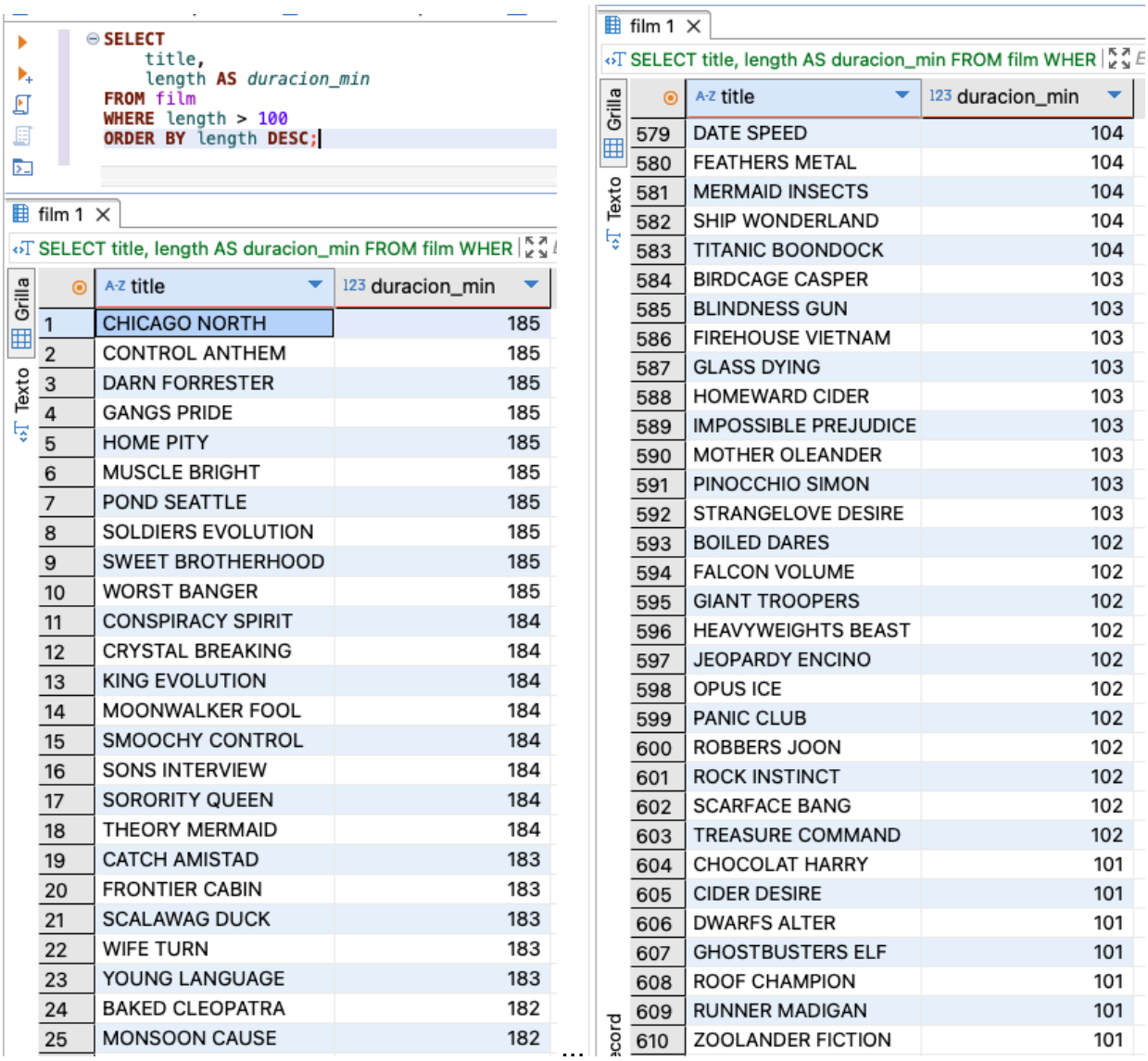
	first_name	last_name
173	DAN	TORN
174	KENNETH	TORN
175	WALTER	TORN
176	LUCILLE	TRACY
177	RENEE	TRACY
178	HELEN	VOIGHT
179	DARYL	WAHLBERG
180	NICK	WAHLBERG
181	BELA	WALKEN
182	ALEC	WAYNE
183	CHRISTOPHER	WEST
184	REESE	WEST
185	GROUCHO	WILLIAMS
186	MORGAN	WILLIAMS
187	SEAN	WILLIAMS
188	BEN	WILLIS
189	GENE	WILLIS
190	HUMPHREY	WILLIS
191	WILL	WILSON
192	FAY	WINSLET
193	RIP	WINSLET
194	ANGELA	WITHERSPOON
195	FAY	WOOD
196	UMA	WOOD
197	CAMERON	WRAY
198	CAMERON	ZELLWEGER
199	JULIA	ZELLWEGER
200	MINNIE	ZELLWEGER

Ejercicio 2 (intermedio) — Películas con duración > 100 minutos

```
SELECT
    title,
    length AS duracion_min
FROM film
```

```
WHERE length > 100
ORDER BY length DESC;
```

Resultado:



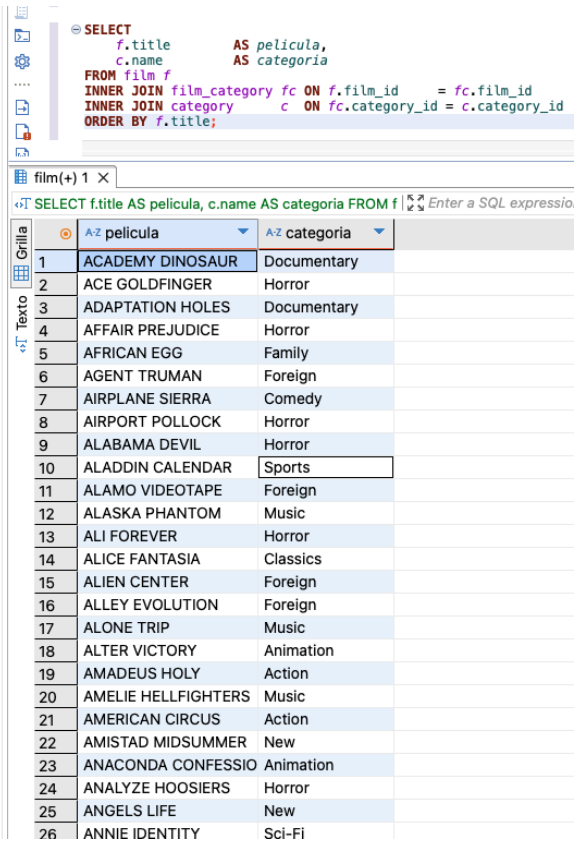
Ejercicio 3 (intermedio) — Película + categoría (JOIN)

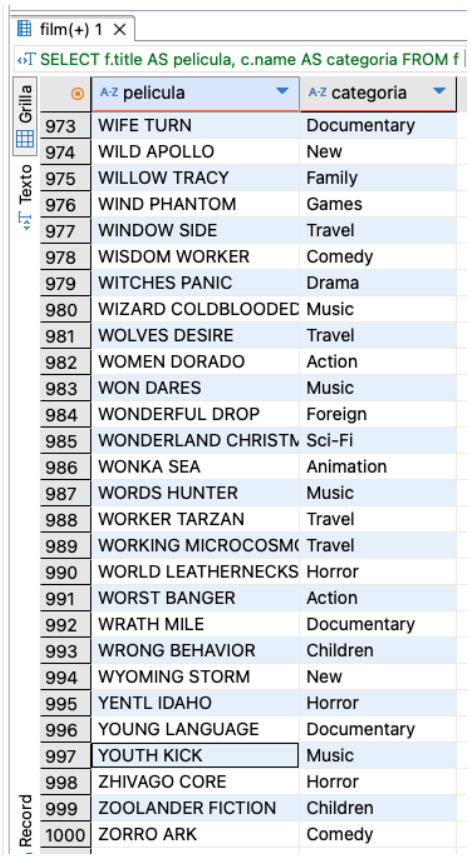
La relación `film` ↔ `category` es muchos-a-muchos a través de la tabla pivot `film_category`.

```
SELECT
    f.title         AS pelicula,
    c.name          AS categoria
FROM film f
INNER JOIN film_category fc ON f.film_id      = fc.film_id
```

```
INNER JOIN category       c  ON fc.category_id = c.category_id
ORDER BY f.title;
```

Resultado: 1 000 filas.





Ejercicio 4 (difícil) — Número de películas por categoría

```
SELECT
    c.name                  AS categoria,
    COUNT(fc.film_id)       AS total_peliculas
FROM category c
INNER JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
ORDER BY total_peliculas DESC, c.name;
```

Resultado:

```sql
SELECT
    c.name                  AS categoria,
    COUNT(fc.film_id)       AS total_peliculas
FROM category c
INNER JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
ORDER BY total_peliculas DESC, c.name;
```

category 1

SELECT c.name AS categoria, COUNT(fc.film_id) AS tot | Enter a SQL expressio

	categoria	total_peliculas
1	Sports	74
2	Foreign	73
3	Family	69
4	Documentary	68
5	Animation	66
6	Action	64
7	New	63
8	Drama	62
9	Games	61
10	Sci-Fi	61
11	Children	60
12	Comedy	58
13	Classics	57
14	Travel	57
15	Horror	56
16	Music	51

Ejercicio 5 (difícil) — Por cliente: total de rentas y total pagado

Combina tres tablas: `customer` (datos del cliente), `rental` (contar rentas) y `payment` (sumar importes). Se usa `COUNT(DISTINCT r.rental_id)` para evitar inflar el conteo si un alquiler tiene más de un pago registrado.

```sql
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name   AS cliente,
    COUNT(DISTINCT r.rental_id)          AS total_rentas,
    ROUND(SUM(p.amount), 2)              AS total_pagado
FROM customer c
INNER JOIN rental  r ON c.customer_id = r.customer_id
INNER JOIN payment p ON r.rental_id   = p.rental_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_pagado DESC, total_rentas DESC;
```

Resultado (top 5 clientes):

```sql
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name   AS cliente,
    COUNT(DISTINCT r.rental_id)          AS total_rentas,
    ROUND(SUM(p.amount), 2)              AS total_pagado
FROM customer c
INNER JOIN rental  r ON c.customer_id = r.customer_id
INNER JOIN payment p ON r.rental_id   = p.rental_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_pagado DESC, total_rentas DESC;
```

customer 1

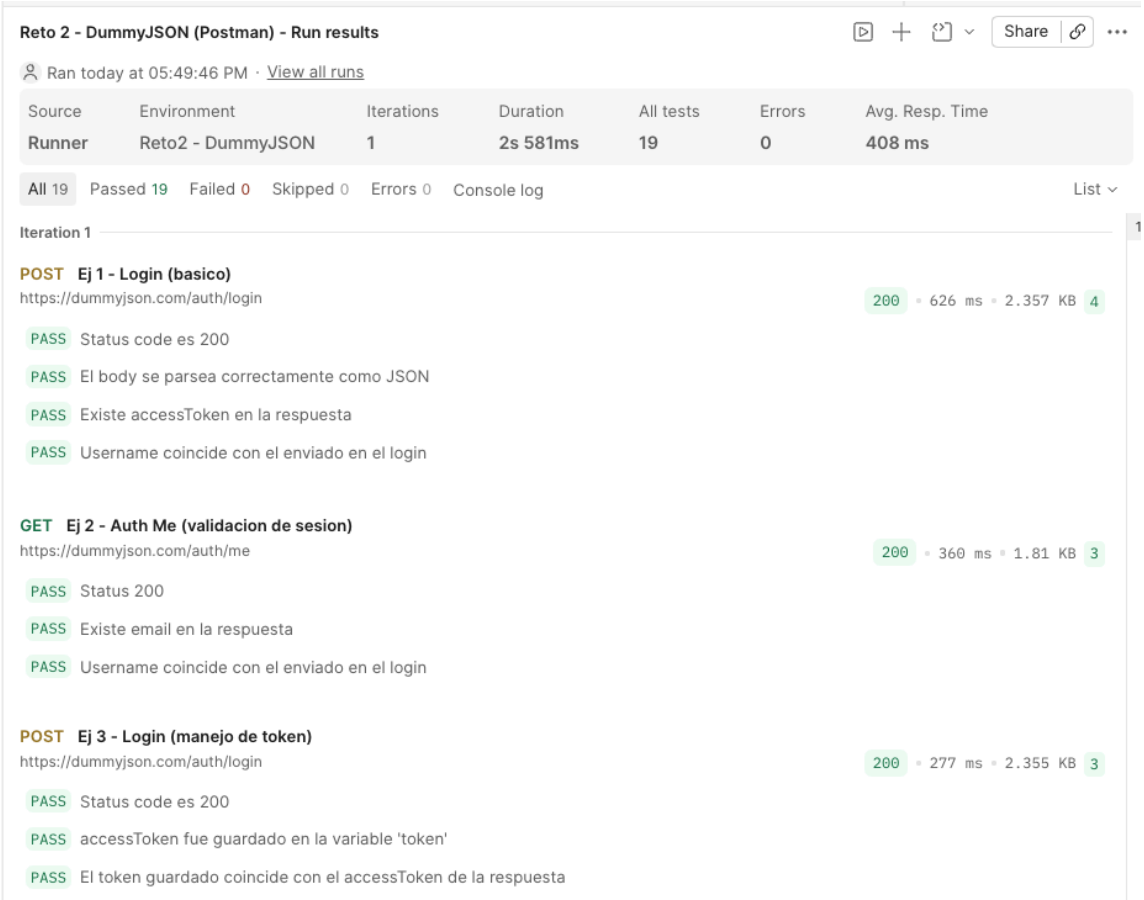
SELECT c.customer_id, c.first_name || ' ' || c.last_name | Enter a SQL expression to filter results (use

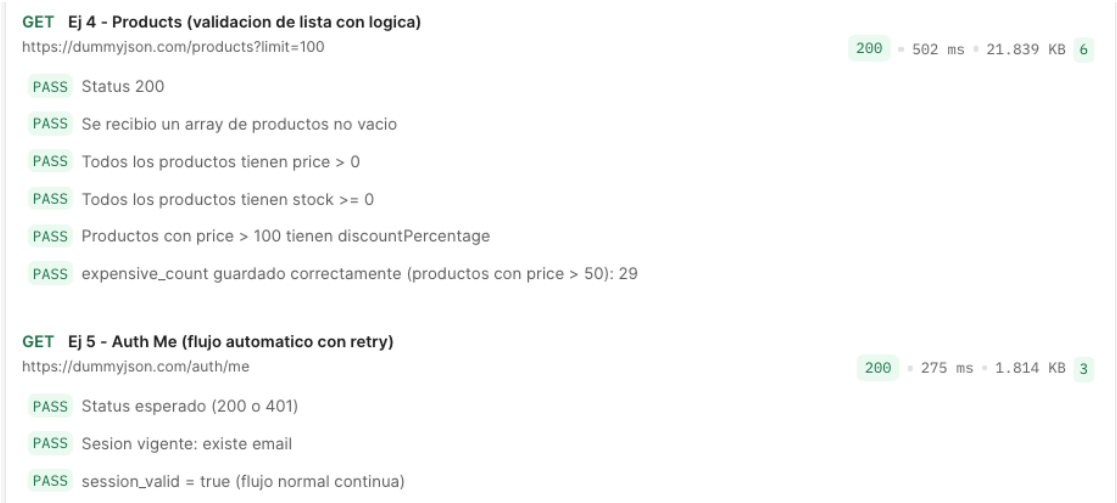
	customer_id	cliente	total_rentas	total_pagado
1	526	KARL SEAL	45	221,55
2	148	ELEANOR HUNT	46	216,54
3	144	CLARA SHAW	42	195,58
4	137	RHONDA KENNEDY	39	194,61
5	178	MARION SNYDER	39	194,61

4. Cómo validar el entregable

4.1 Postman

1. Abrir Postman.
2. **File** → **Import** y seleccionar los dos archivos dentro de `postman/`.
3. En la esquina superior derecha, activar el environment `Reto2 - DummyJSON`.
4. Abrir la colección `Reto 2 - DummyJSON (Postman)` → botón **Run** → **Run collection**.
5. Verificar que los 5 requests se ejecuten en orden y que todos los tests estén en verde.





4.2 SQL

Las queries se pueden validar con cualquier cliente SQL que soporte SQLite. A continuación se describen las dos vías utilizadas durante el desarrollo.

Opción A — DBeaver (cliente gráfico, utilizado para el desarrollo)

- Abrir **DBeaver**.
- **Database** → **New Database Connection** → buscar **SQLite** → **Next**.
- En **Path**, seleccionar el archivo `sql/sakila.db` incluido en el entregable.
- Si es la primera vez, DBeaver ofrece descargar los drivers de SQLite → aceptar.
- **Test Connection…** → debe indicar *Connected* → **Finish**.
- En el **Database Navigator** expandir la conexión → **Tables** para ver el schema (`actor`, `film`, `category`, `film_category`, `customer`, `rental`, `payment`, etc.).
- Abrir un **SQL Editor** (`Database → SQL Editor → New SQL Editor`) o abrir directamente el archivo `sql/reto2-sakila.sql`.
- Asegurarse de que la conexión `sakila` esté seleccionada en el dropdown superior del editor.
- Ejecutar:
  - **Cmd + Enter** → ejecuta la query donde está el cursor (útil para ver cada ejercicio por separado).
  - **Cmd + Alt + Enter** → ejecuta el script completo (las 5 queries). DBeaver abre una pestaña de resultados por cada `SELECT`.

Los resultados se pueden exportar a CSV, Excel, HTML o Markdown con click derecho sobre la grilla → **Export data…**, útil para dejar evidencia adjunta.

Opción B — Terminal (`sqlite3`, cero instalación)

`sqlite3` viene preinstalado en macOS. Desde la carpeta `sql/`:

```
# Ejecutar todas las queries en una corrida
sqlite3 sakila.db < reto2-sakila.sql

# O modo interactivo con formato de tabla
sqlite3 sakila.db
sqlite> .headers on
sqlite> .mode column
sqlite> .read reto2-sakila.sql
```

4.3 Validación cruzada

En la raíz del entregable se incluye este informe y un `README.md` con los mismos pasos abreviados, para que el evaluador pueda verificar sin depender del informe largo.

5. Conclusiones

El reto permitió poner en práctica, de forma integrada, dos habilidades centrales del rol de QA Automation:

- **Lado cliente (API):** cómo construir una colección Postman profesional, con variables, tests, lógica condicional y flujo controlado entre requests. El caso del retry del Ej 5 es especialmente relevante porque refleja un escenario real: qué hacer cuando un token expira en medio de un flujo automatizado.
- **Lado datos (SQL):** cómo consultar una base relacional real (Sakila), comenzando con proyecciones simples y escalando a agregaciones sobre múltiples tablas relacionadas.

Como valor agregado, se detectó y documentó la **incidencia con las credenciales del enunciado**. Antes que asumir y fallar silenciosamente, se priorizó verificar contra el sistema real, dejar evidencia de la incidencia y proponer la solución compatible, que es el criterio profesional esperado en QA.

El entregable está diseñado para ser **autocontenido y reproducible**: el evaluador puede validar todo con las herramientas que ya trae macOS (`sqlite3`) o con una instalación estándar de Postman, sin configuración adicional.

6. Referencias

- Documentación oficial de DummyJSON: https://dummyjson.com/docs
- Endpoint `/auth`: https://dummyjson.com/docs/auth
- Endpoint `/products`: https://dummyjson.com/docs/products
- Base de datos Sakila (original MySQL): https://dev.mysql.com/doc/sakila/en/

- Port SQLite de Sakila: https://github.com/bradleygrant/sakila-sqlite3
- Postman Sandbox API (`pm.*`): https://learning.postman.com/docs/tests-and-scripts/write-scripts/postman-sandbox-api-reference/
- `pm.execution.setNextRequest`: https://learning.postman.com/docs/running-collections/building-workflows/